AEROFLOTA

GESTIÓN DE FLOTA Y MANTENIMIENTO

Memoria técnica del desarrollo · ABAP Cloud

Ficha del proyecto	Información
Código de referencia	ASAR-GFM-2026
Paquete ABAP	ZAEROFLOTA_20
Entorno de desarrollo	ABAP Development Tools (ADT)
Tecnología principal	ABAP Cloud · Programación orientada a objetos
Versión de la memoria	1.0 · 11 de noviembre de 2026
Naturaleza	Proyecto formativo basado en un escenario ficticio

Índice

- 1. Resumen ejecutivo
- 2. Contexto y objetivos
- 3. Alcance y límites
- 4. Entorno y tecnologías
- 5. Diseño y arquitectura de clases
- 6. Funcionalidades implementadas
- 7. Conceptos ABAP y decisiones técnicas
- 8. Pruebas unitarias e integración
- 9. Matriz de trazabilidad funcional
- 10. Resultados y valoración
- 11. Limitaciones y posibles evoluciones
- 12. Anexo: relación de capturas

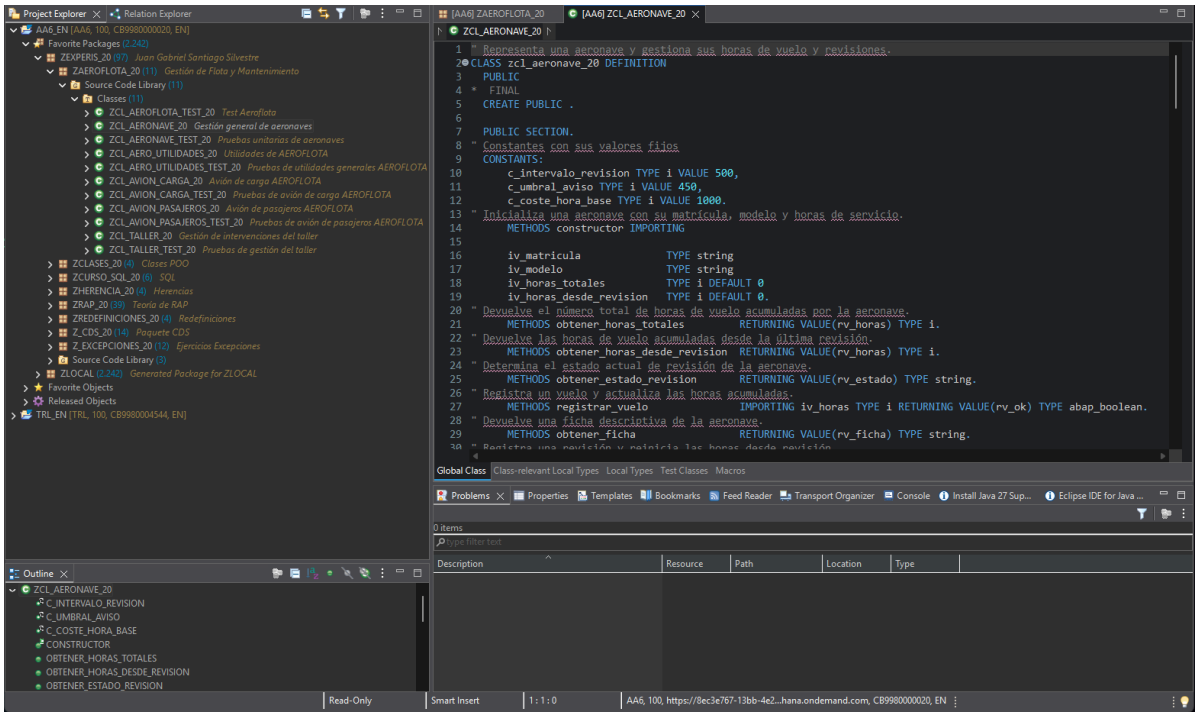
Nota: los recuadros amarillos indican dónde insertar las capturas. Sustituye cada recuadro por una imagen tomada de tu propio entorno ADT. Procura que se lean el nombre del objeto, el código o la salida relevante.

1. Resumen ejecutivo

AEROFLOTA es un proyecto formativo de desarrollo con ABAP Cloud cuyo objetivo es construir un núcleo de lógica reutilizable para gestionar una flota ficticia de aeronaves y sus operaciones básicas de mantenimiento. La solución modela aeronaves genéricas, aviones de pasajeros, aviones de carga, utilidades comunes y un taller que registra intervenciones.

El trabajo se ha desarrollado en SAP ABAP Development Tools (ADT), aplicando programación orientada a objetos: clases globales, encapsulación, constructores, miembros estáticos, herencia, redefinición de métodos, tablas internas y control de flujo. Se han implementado pruebas de las clases y una clase de integración que ejercita conjuntamente las funcionalidades principales.

La entrega actual corresponde a un núcleo de lógica ejecutado en memoria. No incluye persistencia en base de datos ni una interfaz de usuario SAP Fiori; estas capacidades quedan identificadas como posibles evoluciones posteriores mediante RAP y servicios OData.



2. Contexto y objetivos

El escenario plantea una aerolínea regional ficticia, Andalus Air Regional, S.A., que necesita controlar las horas de vuelo, el estado de revisión, los costes operativos y las intervenciones de taller de su flota. El proyecto sustituye conceptualmente un control disperso en hojas de cálculo por un modelo de clases ABAP centralizado y reutilizable.

2.1 Objetivos

- Negocio: representar aeronaves y controlar sus horas de vuelo y revisiones.
- Técnico: construir clases cohesionadas, encapsuladas, reutilizables y ampliables.
- Formativo: practicar progresivamente los fundamentos de la programación orientada a objetos en ABAP.

2.2 Identificación del escenario

AEROFLOTA y Andalus Air Regional son elementos ficticios utilizados exclusivamente con fines formativos. Los intervalos, matrículas y costes son datos de ejemplo y no deben interpretarse como parámetros operativos o regulatorios reales.

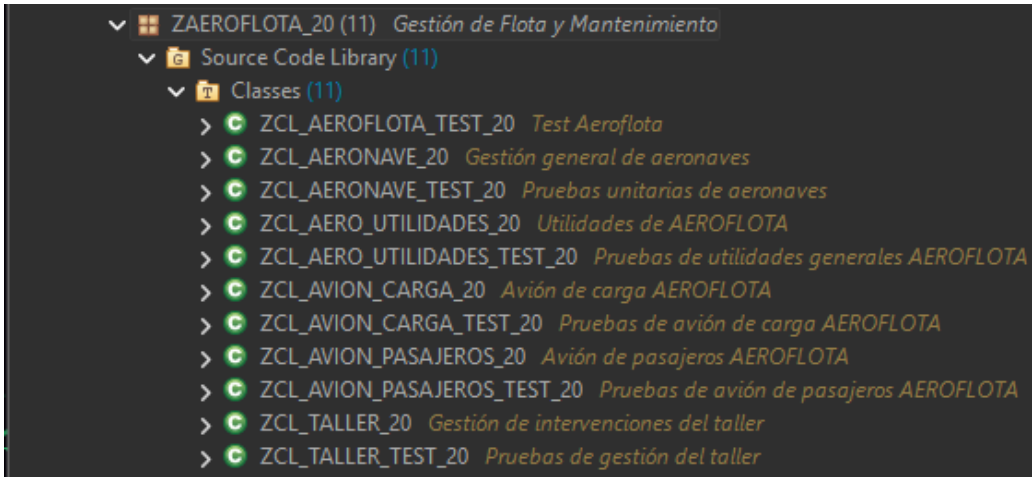
3. Alcance y límites

Incluido en esta entrega	No incluido en esta entrega
Alta y consulta de aeronaves	Persistencia en base de datos
Control de horas y estado de revisión	Interfaz de usuario SAP Fiori
Registro de vuelos y vuelos programados	Servicios OData / API publicados
Cálculo de costes según tipo de aeronave	Integración con sistemas externos
Ocupación de pasajeros y validación de carga	Gestión avanzada de excepciones
Registro y consulta de intervenciones de taller	Planificación por ciclos o calendario

La solución mantiene los datos en memoria durante la ejecución. Por tanto, los objetos y registros creados por las pruebas no constituyen datos persistentes ni una aplicación multiusuario.

4. Entorno y tecnologías

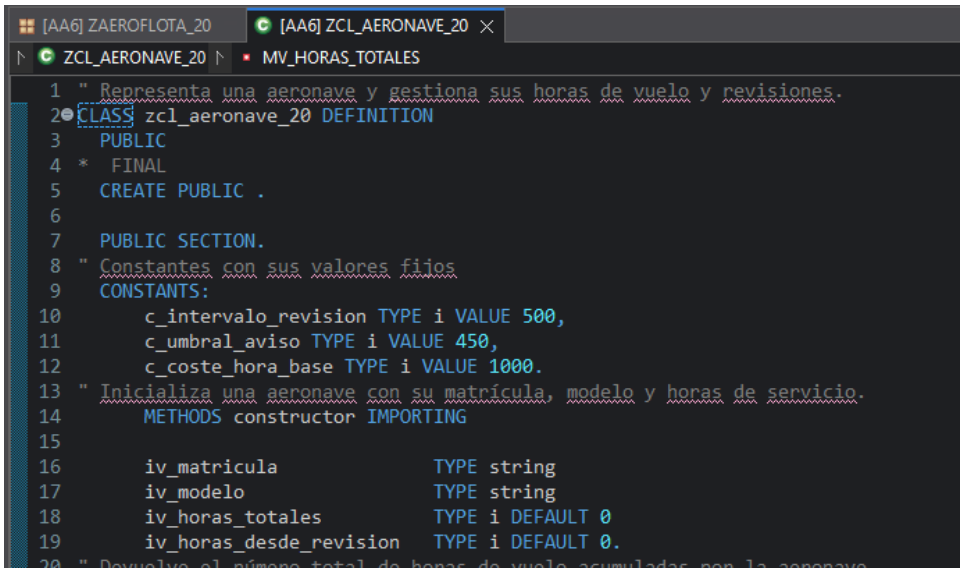
Elemento	Uso en el proyecto
SAP ABAP Cloud	Lenguaje y entorno de desarrollo de la lógica de negocio.
ABAP Development Tools (ADT)	Creación, edición, activación y ejecución de clases.
Clases globales	Organización de las entidades y servicios del dominio.
IF_OO_ADT_CLASSRUN	Ejecución de clases de prueba mediante ADT (F9).
Paquete ZAEROFLOTA_20	Contenedor de los objetos del proyecto.

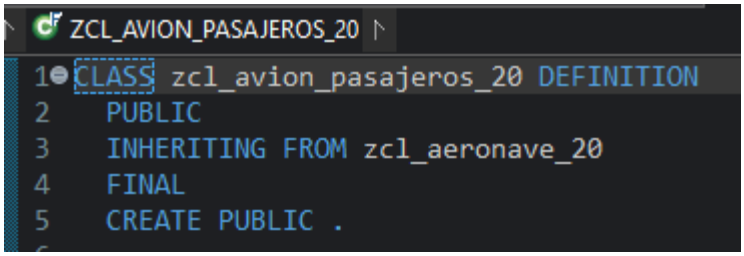


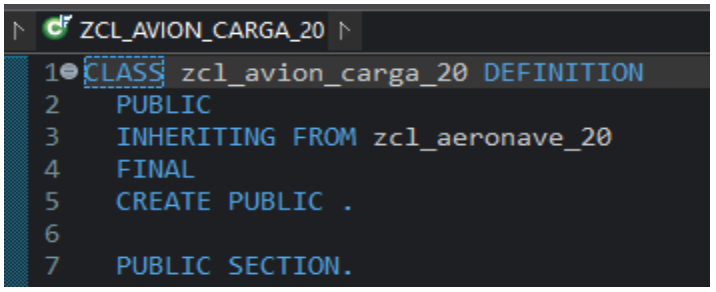
5. Diseño y arquitectura de clases

El modelo contiene cinco clases de lógica. ZCL_AERONAVE_20 actúa como superclase de las dos especializaciones. ZCL_TALLER_20 y ZCL_AERO_UTILIDADES_20 son clases independientes.

Clase	Responsabilidad principal	Relación
ZCL_AERONAVE_20	Datos comunes, horas, vuelos, revisiones, contador y coste base.	Superclase
ZCL_AVION_PASAJEROS_20	Capacidad de pasajeros, ocupación, ficha y coste especializado.	Hereda de ZCL_AERONAVE_20
ZCL_AVION_CARGA_20	Capacidad de carga, validación de mercancía y recargos.	Hereda de ZCL_AERONAVE_20
ZCL_AERO_UTILIDADES_20	Validación de matrículas y conversión de horas a minutos.	Independiente; métodos estáticos
ZCL_TALLER_20	Registro de intervenciones y consultas por matrícula.	Independiente







5.1 Clases de prueba

Clase de prueba	Objeto comprobado
ZCL_AERONAVE_TEST_20	ZCL_AERONAVE_20
ZCL_AVION_PASAJEROS_TEST_20	ZCL_AVION_PASAJEROS_20
ZCL_AVION_CARGA_TEST_20	ZCL_AVION_CARGA_20
ZCL_AERO_UTILIDADES_TEST_20	ZCL_AERO_UTILIDADES_20
ZCL_TALLER_TEST_20	ZCL_TALLER_20
ZCL_AEROFLOTA_TEST_20	Integración de las funcionalidades principales

6. Funcionalidades implementadas

6.1 Gestión general de aeronaves

ZCL_AERONAVE_20 encapsula matrícula, modelo, horas totales y horas desde la última revisión. Ofrece operaciones para consultar la ficha, registrar vuelos, consultar el estado de revisión, registrar una revisión, programar vuelos, estimar vuelos hasta revisión y calcular el coste base.

- Intervalo de revisión configurado: 500 horas desde la última revisión.
- Umbral de aviso: 450 horas.
- Coste base de referencia: 1.000 unidades monetarias por hora.
- El contador estático permite consultar y reiniciar el total de aeronaves creadas.

ZCL_AERONAVE_20 > MV_HORAS_TOTALES

```
  PUBLIC SECTION.
" Constantes con los valores fijos
  CONSTANTS:
      c_intervalo_revision TYPE i VALUE 500,
      c_umbral_aviso TYPE i VALUE 450,
      c_coste_hora_base TYPE i VALUE 1000.
" Inicializa una aeronave con su matrícula, modelo y horas de servicio
      METHODS constructor IMPORTING

      iv_matricula             TYPE string
      iv_modelo                TYPE string
      iv_horas_totales         TYPE i DEFAULT 0
      iv_horas_desde_revision  TYPE i DEFAULT 0.
" Devuelve el número total de horas de vuelo acumuladas por la aeronave.
      METHODS obtener_horas_totales          RETURNING VALUE(rv_horas) TYPE i.
" Devuelve las horas de vuelo acumuladas desde la última revisión.
      METHODS obtener_horas_desde_revision  RETURNING VALUE(rv_horas) TYPE i.
" Determina el estado actual de revisión de la aeronave.
      METHODS obtener_estado_revision        RETURNING VALUE(rv_estado) TYPE string.
" Registra un vuelo y actualiza las horas acumuladas.
      METHODS registrar_vuelo                IMPORTING iv_horas TYPE i RETURNING VALUE(rv_ok) TYPE abap_boolean.
" Devuelve una ficha descriptiva de la aeronave.
      METHODS obtener_ficha                  RETURNING VALUE(rv_ficha) TYPE string.
" Registra una revisión y reinicia las horas desde revisión.
      METHODS registrar_revision.
" Registra vuelos programados hasta alcanzar el límite de revisión.
      METHODS registrar_vuelos_programados   IMPORTING  iv_vuelos TYPE i
                                                        iv_horas_por_vuelo TYPE i
                                                        RETURNING VALUE(rv_registrados) TYPE i.
" Calcula cuántos vuelos pueden realizarse hasta la siguiente revisión.
      METHODS calcular_vuelos_hasta_revision IMPORTING iv_horas_por_vuelo TYPE i
                                                       RETURNING VALUE(rv_num_vuelos) TYPE i.
" Devuelve los datos principales de la aeronave.
      METHODS obtener_datos EXPORTING
                                      ev_matricula TYPE string
                                      ev_modelo TYPE string
                                      ev_horas_totales TYPE i.
" Calcula el coste operativo de un número determinado de horas.
      METHODS calcular_coste_operativo IMPORTING iv_horas TYPE i RETURNING VALUE(rv_coste) TYPE i.
" Devuelve el número total de aeronaves creadas.
      CLASS-METHODS obtener_total_aeronaves RETURNING VALUE(rv_total) TYPE i.
" Reinicia el contador estático de aeronaves.
      CLASS-METHODS reiniciar_contador.

  PROTECTED SECTION.

    DATA:
        mv_matricula TYPE string,
        mv_modelo    TYPE string.

  PRIVATE SECTION.

    DATA:
    mv_horas_totales         TYPE i,
    mv_horas_desde_revision  TYPE i.

    CLASS-DATA gv_total_aeronaves TYPE i.

ENDCLASS.
```

ZCL_AERONAVE_20

```
CLASS zcl_aeronave_20 IMPLEMENTATION.
  METHOD constructor.

    mv_matricula = iv_matricula.
    mv_modelo = iv_modelo.
    mv_horas_totales = iv_horas_totales.
    mv_horas_desde_revision = iv_horas_desde_revision.

    gv_total_aeronaves = gv_total_aeronaves + 1.

  ENDMETHOD.

  METHOD obtener_horas_totales.
" Devuelve las horas totales acumuladas.
    rv_horas = mv_horas_totales.

  ENDMETHOD.

  METHOD obtener_horas_desde_revision.
" Devuelve las horas acumuladas desde la última revisión.
    rv_horas = mv_horas_desde_revision.

  ENDMETHOD.

  METHOD obtener_ficha.
" Construye y devuelve la ficha descriptiva de la aeronave.
    rv_ficha = |Matrícula: { mv_matricula } | &&
               |Modelo: { mv_modelo } | &&
               |Horas totales: { mv_horas_totales } | &&
               |Horas desde revisión: { mv_horas_desde_revision }|.

  ENDMETHOD.
```

6.2 Aviones de pasajeros

ZCL_AVION_PASAJEROS_20 hereda de ZCL_AERONAVE_20 y añade la capacidad de pasajeros. Redefine la ficha para incluir el número de plazas y el cálculo del coste operativo para añadir el coste específico de cabina. También calcula el porcentaje de ocupación, limitándolo al 100 %.

- Ejemplo comprobado: 120 pasajeros de 150 plazas equivalen al 80 % de ocupación.
- Ejemplo comprobado: 200 pasajeros para 150 plazas se limita al 100 %.
- Coste de cabina: 150 por hora; para 10 horas, el coste total probado es 11.500.

ZCL_AVION_PASAJEROS_20

```
8
9  " Recargo de cabina por cada hora de vuelo, expresado en euros.
10   CONSTANTS c_coste_cabina TYPE i VALUE 150.
11
12   METHODS constructor
13   IMPORTING
14     iv_matricula TYPE string
15     iv_modelo TYPE string
16     iv_horas_totales TYPE i DEFAULT 0
17     iv_horas_desde_revision TYPE i DEFAULT 0
18     iv_capacidad_pasajeros TYPE i.
19
20   METHODS obtener_capacidad_pasajeros RETURNING VALUE(rv_capacidad) TYPE i.
21
22   METHODS calcular_coste_operativo REDEFINITION.
23
24   METHODS obtener_ficha REDEFINITION.
25
26 " Calcula el porcentaje de ocupación del avión.
27   METHODS calcular_ocupacion IMPORTING iv_pasajeros TYPE i RETURNING VALUE(rv_porcentaje) TYPE i.
28
29   PROTECTED SECTION.
30   PRIVATE SECTION.
31
32    DATA mv_capacidad_pasajeros TYPE i.
33
34 ENDCLASS.
35
36
```

6.3 Aviones de carga

ZCL_AVION_CARGA_20 hereda de la superclase y añade la capacidad máxima de carga. Valida que los kilogramos sean positivos y no superen la capacidad configurada. Calcula recargos según el tipo de mercancía y redefine el coste operativo.

- Coste de bodega: 250 por hora; para 10 horas, el coste total probado es 12.500.
- Tipos de mercancía contemplados: G (general), P (perecedera) y M (mercancía especial).
- Se comprobó la aceptación de 15.000 kg y el rechazo de 20.001 kg para una capacidad de 20.000 kg en integración.

ZCL_AVION_CARGA_20

```
  PUBLIC SECTION.

" Recargo de bodega por cada hora de vuelo, expresado en euros.
    CONSTANTS c_coste_bodega TYPE i VALUE 250.
    TYPES ty_tipo_mercancia TYPE c LENGTH 1.
  METHODS constructor
  IMPORTING
    iv_matricula TYPE string
    iv_modelo TYPE string
    iv_horas_totales TYPE i DEFAULT 0
    iv_horas_desde_revision TYPE i DEFAULT 0
    iv_capacidad_carga TYPE i.

  METHODS obtener_capacidad_carga RETURNING VALUE(rv_capacidad) TYPE i.

  METHODS calcular_coste_operativo REDEFINITION.

" Calcula el recargo según el tipo de mercancía.
  METHODS calcular_recargo_mercancia IMPORTING iv_tipo TYPE ty_tipo_mercancia RETURNING VALUE(rv_recargo) TYPE i.

" Comprueba que la carga esté dentro de los límites permitidos.
  METHODS validar_carga IMPORTING iv_kg TYPE i RETURNING VALUE(rv_valida) TYPE abap_boolean.

  PROTECTED SECTION.
  PRIVATE SECTION.

   DATA mv_capacidad_carga TYPE i.

ENDCLASS.
```

6.4 Utilidades comunes

ZCL_AERO_UTILIDADES_20 concentra métodos estáticos que no requieren crear una aeronave: validación de matrículas y conversión de horas a minutos.

- La validación comprueba la longitud y el prefijo EC-.
- Conversión comprobada: 2 horas equivalen a 120 minutos; 0 horas equivalen a 0 minutos.

ZCL_AERO_UTILIDADES_20

```
CLASS zcl_aero_utilidades_20 DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.

*   Guardamos las 3 constantes que utilizaremos más adelante
      CONSTANTS:
*   Prefijo de matrícula
    c_prefijo_espana TYPE string VALUE 'EC-',
*    Longitud de matrícula
    c_longitud_matricula TYPE i VALUE 6,
*    Paso de horas a minutos
    c_minutos_por_hora TYPE i VALUE 60.


*   Declaramos los metodos estaticos

    CLASS-METHODS validar_matricula IMPORTING iv_matricula TYPE string RETURNING VALUE(rv_valida) TYPE abap_boolean

    CLASS-METHODS convertir_horas_a_minutos IMPORTING iv_horas TYPE i RETURNING VALUE(rv_minutos) type i.

  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
```

6.5 Taller y mantenimiento

ZCL_TALLER_20 mantiene una tabla interna de intervenciones. Cada registro contiene matrícula, tipo de intervención y horas. Permite consultar las horas acumuladas y contar las intervenciones de una matrícula concreta.

- EC-ABC: 10 horas y 3 intervenciones.
- EC-XYZ: 4 horas y 1 intervención.
- Matrícula sin intervenciones: 0 horas y 0 registros.

ZCL_TALLER_20

```
CLASS zcl_taller_20 DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.

  TYPES ty_tipo_intervencion TYPE c LENGTH 1.

  " Estructura que representa una intervención de mantenimiento.
  TYPES:
    BEGIN OF ty_intervencion,
      matricula TYPE string,
      tipo      TYPE ty_tipo_intervencion,
      horas     TYPE i,
    END OF ty_intervencion.

  " Tabla interna que almacena todas las intervenciones.
  TYPES ty_intervenciones TYPE STANDARD TABLE OF ty_intervencion
    WITH EMPTY KEY.

  " Inicializa el taller con su nombre.
  METHODS constructor IMPORTING iv_nombre TYPE string.


  " Registra una intervención de mantenimiento de una aeronave.
  METHODS registrar_intervencion IMPORTING
                                       iv_matricula TYPE string
                                       iv_tipo      TYPE ty_tipo_intervencion
                                       iv_horas     TYPE i.

  " Consulta las horas acumuladas de intervención de una aeronave.
  METHODS consultar_horas_matricula IMPORTING iv_matricula TYPE string RETURNING VALUE(rv_horas) TYPE i.

  " Cuenta las intervenciones registradas para una aeronave.
  METHODS contar_intervenciones IMPORTING iv_matricula TYPE string RETURNING VALUE(rv_total) TYPE i.
```

7. Conceptos ABAP y decisiones técnicas

Concepto	Aplicación en AEROFLOTA
Clases y objetos	Clases globales y creación de instancias mediante NEW.
Encapsulación	Atributos PRIVATE y PROTECTED; acceso mediante métodos.
Constructores	Inicialización de atributos y uso de super->constructor en subclases.
Miembros estáticos	Contador de aeronaves y métodos de utilidades.
Herencia	Pasajeros y carga heredan de la clase aeronave.
Redefinición	Las subclases amplían ficha y coste operativo usando super-> cuando corresponde.
Control de flujo	IF/ELSEIF, CASE, DO/EXIT y WHILE.
Tablas internas	Registro de intervenciones del taller con APPEND VALUE y LOOP AT.
Parámetros	Prefijos iv_ para entrada, ev_ para salidas múltiples y rv_ para retorno.

Convención de nomenclatura del proyecto: las clases llevan el prefijo ZCL_ y el sufijo _20; las variables locales utilizan lv_ y las constantes c_.

```
METHOD constructor.

  super->constructor( iv_matricula = iv_matricula iv_modelo = iv_modelo iv_horas_totales = iv_horas_totales iv_horas_desde_revision = iv_horas_desde_revision ).
   mv_capacidad_pasajeros = iv_capacidad_pasajeros.

ENDMETHOD.
```

```
METHODS calcular_coste_operativo REDEFINITION.
```

```
METHOD calcular_coste_operativo.
" Calcula el coste base más el recargo de cabina.

  rv_coste = super->calcular_coste_operativo( iv_horas ).

  rv_coste = rv_coste + ( iv_horas * c_coste_cabina ).
```

8. Pruebas unitarias e integración

Las pruebas individuales se ejecutaron desde ADT mediante F9 en las clases de ejecución IF_OO_ADT_CLASSRUN. Durante el desarrollo se confirmaron resultados correctos para las pruebas de aeronave, pasajeros, carga, utilidades y taller.

Prueba	Casos/resultados representativos
Aeronave	Estados de revisión, vuelos válidos e inválidos, revisión, vuelos programados, estimación y contador.
Pasajeros	Ficha con plazas, coste de 10 h = 11.500 y ocupación 80 %, 100 % y 0 %.
Carga	Coste de 10 h = 12.500, validación de capacidad y recargos por tipo de mercancía.
Utilidades	EC-ABC válida; EC-AB, XX-ABC y ECABCD inválidas; conversión 2 h = 120 min.
Taller	EC-ABC: 10 h/3 intervenciones; EC-XYZ: 4 h/1; EC-999: 0/0.

```
METHOD if_oo_adt_classrun~main.

    DATA(lo_avion) = NEW zcl_avion_pasajeros_20(
    iv_matricula = 'EC-PAX'
    iv_modelo = 'AIRBUS A320'
    iv_horas_totales = 12000
    iv_horas_desde_revision = 100
    iv_capacidad_pasajeros = 180
      ).

" Creacion avión de pasajeros
   out->write( 'Avion de pasajeros creado correctamente' ).
" Obtener capacidad de pasajeros
   out->write( lo_avion->obtener_capacidad_pasajeros( ) ).
" Ficha heredada de métodos padres
   out->write( lo_avion->obtener_ficha( ) ).
" Obtener estado de método padre
   out->write( lo_avion->obtener_estado_revision( ) ).
" Registramos 50 horas
   out->write( lo_avion->registrar_vuelo( 50 ) ).
" Obtenemos horas totales para ver la suma las 50 h anteriores
   out->write( lo_avion->obtener_horas_totales( ) ).
" Obtenemos horas de revisión totales
   out->write( lo_avion->obtener_horas_desde_revision( ) ).
" Coste avión pasajeros
   out->write( lo_avion->calcular_coste_operativo( 10 ) ).
" Comprobación de ocupantes en el avión en porcentaje
   out->write( |Ocupacion 120 pasajeros: {
     lo_avion->calcular_ocupacion( 120 ) } %| ).

   out->write( |Ocupacion 200 pasajeros: {
     lo_avion->calcular_ocupacion( 200 ) } %| ).

   out->write( |Ocupacion 0 pasajeros: {
     lo_avion->calcular_ocupacion( 0 ) } %| ).


ENDMETHOD.
```

```
Global Class | Class-relevant Local Types | Local Types | Test Classes | Macros
Problems | Properties | Templates | Bookmarks | Feed Reader | Transport Organizer | Console | Install J

ABAP Console
Avion de pasajeros creado correctamente
180
Matrícula: EC-PAX Modelo: AIRBUS A320 Horas totales: 12000 Horas desde revisión: 100 - 180 plazas
OPERATIVA
X
12050
150
11500
Ocupacion 120 pasajeros: 67 %
Ocupacion 200 pasajeros: 100 %
Ocupacion 0 pasajeros: 0 %
```

8.1 Prueba de integración

ZCL_AEROFLOTA_TEST_20 integra objetos de pasajeros, carga y taller. En su ejecución se comprobaron la creación de objetos, las fichas, el registro de vuelos, los costes, las intervenciones, las utilidades, la ocupación y la validación de carga.

Comprobación integrada	Resultado confirmado
Vuelo de 10 horas en aeronave de pasajeros y de carga	Registrado en ambos objetos
Coste de pasajeros / carga	11.500 / 12.500
Mantenimiento EC-PAX	8 horas y 2 intervenciones
Mantenimiento EC-CGO	4 horas y 1 intervención
Matrículas EC-ABC / XX-ABC	Válida / no válida
Conversión de 2 horas	120 minutos
Ocupación de pasajeros	80 % y 100 % en los casos probados
Carga de 15.000 kg / 20.001 kg	Aceptada / rechazada

ZCL_AEROFLOTA_TEST_20 ▸ IF_OO_ADT_CLASSRUN~MAIN

```
CLASS zcl_aeroflota_test_20 IMPLEMENTATION.


  METHOD if_oo_adt_classrun~main.

    out->write( 'AEROFLOTA: entorno preparado' ).

    " Creamos un avión de pasajeros.
    DATA(lo_pasajeros) = NEW zcl_avion_pasajeros_20(
      iv_matricula = 'EC-PAX'
      iv_modelo = 'AIRBUS A320'
      iv_horas_totales = 0
      iv_horas_desde_revision = 0
      iv_capacidad_pasajeros = 150
    ).

    " Creamos un avión de carga.
    DATA(lo_carga) = NEW zcl_avion_carga_20(
      iv_matricula = 'EC-CGO'
      iv_modelo = 'BOEING B737F'
      iv_horas_totales = 0
      iv_horas_desde_revision = 0
      iv_capacidad_carga = 20000
    ).

    " Creamos el taller de mantenimiento.
    DATA(lo_taller) = NEW zcl_taller_20(
      iv_nombre = 'Taller AEROFLOTA'
    ).

    " Mostramos las fichas de las aeronaves.
    out->write( lo_pasajeros->obtener_ficha( ) ).
    out->write( lo_carga->obtener_ficha( ) ).

    " Confirmamos que el taller se ha creado.
    out->write( 'Taller AEROFLOTA creado correctamente' ).

    " Registramos un vuelo de 10 horas en el avión de pasajeros.
    lo_pasajeros->registrar_vuelo( 10 ).

    " Registramos un vuelo de 10 horas en el avión de carga.
    lo_carga->registrar_vuelo( 10 ).

    " Consultamos el coste operativo del avión de pasajeros.
    out->write( |Coste pasajeros: {
      lo_pasajeros->calcular_coste_operativo( 10 ) } EUR| ).

    " Consultamos el coste operativo del avión de carga.
    out->write( |Coste carga: {
      lo_carga->calcular_coste_operativo( 10 ) } EUR| ).
```

ABAP Console

```
AEROFLOTA: entorno preparado
Matrícula: EC-PAX Modelo: AIRBUS A320 Horas totales: 0 Horas desde revisión: 0 - 150 plazas
Matrícula: EC-CGO Modelo: BOEING B737F Horas totales: 0 Horas desde revisión: 0
Taller AEROFLOTA creado correctamente
Coste pasajeros: 11500 EUR
Coste carga: 12500 EUR
```

9. Matriz de trazabilidad funcional

ID	Descripción	Clase(s)	Método(s)	Estado
RF-01	Alta de aeronave	ZCL_AERONAVE_20	Constructor	Probado
RF-02	Consultar ficha	ZCL_AERONAVE_20; ZCL_AVION_PASAJEROS_20	obtener_ficha	Probado
RF-03	Registrar vuelo con validación	ZCL_AERONAVE_20	registrar_vuelo	Probado
RF-04	Consultar estado de revisión	ZCL_AERONAVE_20	obtener_estado_revision	Probado
RF-05	Registrar revisión	ZCL_AERONAVE_20	registrar_revision	Probado
RF-06	Registrar vuelos programados	ZCL_AERONAVE_20	registrar_vuelos_programados	Probado
RF-07	Estimar vuelos hasta revisión	ZCL_AERONAVE_20	calcular_vuelos_hasta_revision	Probado
RF-08	Contar aeronaves	ZCL_AERONAVE_20	obtener_total_aeronaves; reiniciar_contador	Probado
RF-09	Validar matrícula y convertir horas	ZCL_AERO_UTILIDADES_20	validar_matricula; convertir_horas_a_minutos	Probado
RF-10	Calcular coste por tipo	ZCL_AERONAVE_20; ZCL_AVION_PASAJEROS_20; ZCL_AVION_CARGA_20	calcular_coste_operativo	Probado
RF-11	Calcular ocupación	ZCL_AVION_PASAJEROS_20	calcular_ocupacion	Probado
RF-12	Validar carga y recargo	ZCL_AVION_CARGA_20	validar_carga; calcular_recargo_mercancia	Probado
RF-13	Registrar y consultar intervenciones	ZCL_TALLER_20	registrar_intervencion; consultar_horas_matricula; contar_intervenciones	Probado

La matriz vincula cada requisito funcional con el elemento ABAP responsable y la comprobación realizada. Los estados reflejan las ejecuciones confirmadas durante el desarrollo.

9.1 Requisitos técnicos

ID	Criterio	Aplicación / estado
RT-01	Clases globales y clases de prueba	Clases globales creadas; tests individuales y de integración.
RT-02	Encapsulación	Atributos privados/protegidos y acceso mediante métodos.
RT-03	Herencia	Superclase no FINAL; subclases FINAL.
RT-04	Convención de parámetros	Prefijos iv_, ev_ y rv_ aplicados en las firmas.
RT-05	Sin acceso a base de datos	Lógica ejecutada en memoria, sin persistencia.
RT-06	Comentarios de propósito	Comentarios incorporados durante el desarrollo; comprobar cobertura total en la revisión final.

10. Resultados y valoración

El desarrollo ha permitido construir un modelo orientado a objetos con una jerarquía de aeronaves y dos clases de especialización, junto con utilidades y gestión básica de taller. Las pruebas individuales y la integración confirmaron los comportamientos principales descritos en los requisitos funcionales.

El proyecto demuestra el uso práctico de encapsulación, herencia, redefinición, constructores, métodos estáticos, tablas internas y pruebas de ejecución en ADT. La separación de responsabilidades facilita ampliar el modelo en una siguiente iteración.

11. Limitaciones y posibles evoluciones

La entrega actual es deliberadamente un núcleo de lógica en memoria. Para evolucionarlo hacia una aplicación empresarial completa, se podrían abordar las siguientes ampliaciones:

- Persistencia de aeronaves, vuelos e intervenciones mediante tablas de base de datos.
- Modelado de datos con CDS View Entities.
- Definición de comportamientos transaccionales mediante RAP.
- Publicación de servicios OData.
- Construcción de una interfaz SAP Fiori Elements.
- Gestión de autorizaciones, validaciones y mensajes de aplicación adecuados al escenario.

Estas ampliaciones no forman parte de la entrega descrita en esta memoria y no deben presentarse como funcionalidades ya implementadas.

12. Anexo: relación de capturas

Lista de comprobación para completar el documento antes de exportarlo a PDF:

- ☐ Vista del paquete ZAEROFLOTA_20 con los objetos principales.
- ☐ Entorno ADT conectado, sin mostrar datos sensibles.
- ☐ Definición de la superclase y las relaciones de herencia.
- ☐ Definición/implementación de ZCL_AERONAVE_20.
- ☐ Implementación de la clase de pasajeros.
- ☐ Implementación de la clase de carga.
- ☐ Implementación de utilidades.
- ☐ Implementación de taller.
- ☐ Salidas F9 de pruebas individuales.
- ☐ Salida F9 de la prueba de integración.